BLG307-VERİTABANI YÖNETİM SİSTEMLERİ DÖNEM PROJESİ RAPORU

MUSTAFA TALHA KARAGENÇ

231201012

RumeliCare Poliklinik Yönetim Sistemi Veritabanı Projesi

Ders: Veritabanı Yönetim Sistemleri **Öğrenci:** Mustafa Talha Karagenç **Numara:** 231201012

1. Giriş ve Proje Tanımı

RumeliCare Polikliniği, günlük ortalama 150 hasta kapasitesiyle İstanbul'da hizmet veren ve manuel kayıt sisteminden dijitalleşme sürecine geçen orta ölçekli bir sağlık kuruluşudur. Bu proje kapsamında, kliniğin hasta takibi, randevu yönetimi, muayene süreçleri ve finansal operasyonlarını yönetecek, veri bütünlüğünü ve güvenliğini sağlayan ilişkisel bir veritabanı sistemi tasarlanmıştır.

Projenin temel amacı; Dahiliye, Kardiyoloji, Ortopedi, Dermatoloji, Göz ve KBB olmak üzere 6 farklı branşta hizmet veren kliniğin operasyonel verimliliğini artırmak, hatalı randevu veya fatura girişlerini engellemek ve geçmişe dönük veri takibini kolaylaştırmaktır.

2. Veritabanı Tasarımı ve Normalizasyon

Sistem, veri tekrarını önlemek ve tutarlılığı sağlamak amacıyla **3. Normal Form (3NF)** kurallarına uygun olarak tasarlanmıştır. Veritabanı şeması aşağıdaki temel yapı taşlarından oluşmaktadır:

2.1. Temel Tablolar

- **Hastalar ve Doktorlar:** Kişisel bilgiler, iletişim detayları ve (hastalar için) alerji bilgileri ayrı tablolarda tutularak yönetim kolaylaştırılmıştır. Doktorlar, **Bolumler** tablosu ile ilişkilendirilmiştir.
- **Randevu ve Muayene Yönetimi:** Randevu sistemi, hastaların doktorlarla eşleştiği temel tablodur. Muayeneler tablosu ise randevularla birebir ilişki (1:1) kurarak teşhis ve tedavi notlarını saklar.
- **Reçete ve İlaç Sistemi:** Bir reçetenin birden fazla ilacı barındırabilmesi için **Receteler** (üst bilgi) ve **ReceteDetay** (ilaç ve dozaj bilgisi) tabloları ayrılarak **Ilaclar** kataloğu ile ilişkilendirilmiştir.
- **Finansal Yönetim:** Her muayene sonucunda oluşan mali kayıtlar **Faturalar** tablosunda tutulmaktadır.

```sql
--  TABLOLARIN OLUŞTURULMASI
```

```sql
CREATE TABLE Bolumler (
    BolumID INT IDENTITY(1,1) PRIMARY KEY,
    Ad NVARCHAR(50) NOT NULL,
    Kat INT,
    CalismaSaatleri NVARCHAR(50)
);
```

3. Geliştirilen Veritabanı Nesneleri

Sistemin iş mantığını (Business Logic) veritabanı seviyesinde yönetmek için Stored Procedure (Saklı Yordamlar) ve Trigger (Tetikleyiciler) mekanizmaları kullanılmıştır.

3.1. Saklı Yordamlar (Stored Procedures)

Sistemin rutin operasyonlarını standartlaştırmak için aşağıdaki prosedürler geliştirilmiştir:

- **sp_RandevuOlustur:** Hasta ve doktor uygunluğunu kontrol ederek yeni randevu kaydı oluşturur.
- **sp_MuayeneTamamla:** Randevuyu tamamlandı statüsüne çeker, teşhis bilgilerini girer ve süreci sonlandırır.
- **sp_ReceteOlustur:** Muayene sonrası hastalara ilaç reçete edilmesini sağlar.
- **sp_GunlukRapor & sp_HastaGecmisi:** Yöneticiler ve doktorlar için raporlama ekranlarına veri sağlayan prosedürlerdir.

```sql
-- Stored Procedures
CREATE PROCEDURE sp_RandevuOlustur @HastaID INT, @DoktorID INT, @Tarih DATE, @Saat TIME AS
BEGIN
    INSERT INTO Randevular (HastaID, DoktorID, Tarih, Saat) VALUES (@HastaID, @DoktorID, @Tarih, @Saat);
END;
GO

CREATE PROCEDURE sp_MuayeneTamamla @RandevuID INT, @Sikayet NVARCHAR(MAX), @Teshis NVARCHAR(MAX), @TedaviNotu NVARCHAR(MAX) AS
BEGIN
    UPDATE Randevular SET Durum = 'Tamamlandı' WHERE RandevuID = @RandevuID;
    INSERT INTO Muayeneler (RandevuID, Sikayet, Teshis, TedaviNotu) VALUES (@RandevuID, @Sikayet, @Teshis, @TedaviNotu);
END;
GO

CREATE PROCEDURE sp_ReceteOlustur @MuayeneID INT, @IlacID INT, @Dozaj NVARCHAR(50), @Talimat NVARCHAR(255) AS
BEGIN
    DECLARE @ReceteID INT;
    SELECT @ReceteID = ReceteID FROM Receteler WHERE MuayeneID = @MuayeneID;
    IF @ReceteID IS NULL BEGIN INSERT INTO Receteler (MuayeneID) VALUES (@MuayeneID); SET @ReceteID = SCOPE_IDENTITY(); END
    INSERT INTO ReceteDetay (ReceteID, IlacID, Dozaj, Talimat) VALUES (@ReceteID, @IlacID, @Dozaj, @Talimat);
END;
GO

CREATE PROCEDURE sp_GunlukRapor @Tarih DATE AS BEGIN SELECT COUNT(*) FROM Randevular WHERE Tarih = @Tarih; END;
GO
CREATE PROCEDURE sp_HastaGecmisi @HastaID INT AS BEGIN SELECT * FROM Randevular WHERE HastaID = @HastaID; END;
GO
```

3.2. Tetikleyiciler (Triggers)

Otomasyonu sağlamak ve hataları önlemek için kritik noktalara tetikleyiciler yerleştirilmiştir:

- trg_RandevuCakismaKontrol (BEFORE INSERT): Aynı doktora, aynı tarih ve saatte ikinci bir randevu verilmesini engelleyerek "Double Booking" sorununu çözer.
- trg_OtomatikFatura (AFTER INSERT): Muayene kaydı girildiği anda, doktorun muayene ücretini baz alarak otomatik olarak fatura kaydı oluşturur.
- trg_RandevuIptalLog (AFTER UPDATE): İptal edilen randevuları tespit eder ve IslemLoglari tablosuna tarih/saat etiketiyle kayıt düşerek denetim izi (Audit Trail) oluşturur.
- trg_HastaSonZiyaret: Hasta her muayene olduğunda, hasta kartındaki "Son Ziyaret Tarihi" bilgisini otomatik günceller.

Bazı senaryolar ve testleri

```sql
--randevu iptal etme kontrolü
-- Önce iptal edeceğimiz randevuyu (Örn: ID 45) ve Log tablosunu kontrol edelim
SELECT * FROM Randevular WHERE RandevuID = 45;
SELECT * FROM IslemLoglari;

-- ŞİMDİ RANDEVUYU İPTAL EDİYORUZ
UPDATE Randevular
SET Durum = 'İptal'
WHERE RandevuID = 45;

-- SONUÇLARI KONTROL ET
-- 1. Randevu durumu 'İptal' oldu mu?
SELECT * FROM Randevular WHERE RandevuID = 45;

-- 2. Log tablosuna kayıt düştü mü?
SELECT * FROM IslemLoglari ORDER BY LogID DESC;
```

```sql
-- 1. Hasta Yoksa Ekleme durumu
IF NOT EXISTS (SELECT 1 FROM Hastalar WHERE TCNo = '12345678901')
    INSERT INTO Hastalar (TCNo, AdSoyad, Cinsiyet) VALUES ('12345678908', 'Ayyşe Yılmaz', 'K');

-- 2. Randevuyu Oluştur (ID'leri isme ve TC'ye göre otomatik bulur)
-- Not: Seed verilerinde 12 Ocak dolu olduğu için tarihi 17 Ocak yaptım.
INSERT INTO Randevular (HastaID, DoktorID, Tarih, Saat)
SELECT
    (SELECT HastaID FROM Hastalar WHERE TCNo = '12345678901'),
    (SELECT DoktorID FROM Doktorlar WHERE AdSoyad = 'Dr. İLHAN KALP'),
    '2026-01-29',
    '10:00';

-- Sonucu Göster
SELECT TOP 1 * FROM Randevular ORDER BY RandevuID DESC;
```

```sql
--çakışma kontrolü aynı anda bir doktor iki muayeneye giremez
-- Mevcut dolu randevuyu görelim
SELECT * FROM Randevular WHERE DoktorID = 5 AND Tarih = '2026-01-16' AND Saat = '09:30';

-- AYNI SAATE BAŞKA BİR HASTAYA (Örn: HastaID 1) RANDEVU EKLEMEYE ÇALIŞALIM
-- Bu kodu çalıştırdığında "Messages" kısmında KIRMIZI HATA görmelisin.
BEGIN TRY
    INSERT INTO Randevular (HastaID, DoktorID, Tarih, Saat)
    VALUES (1, 5, '2026-01-16', '09:30');
END TRY
BEGIN CATCH
    SELECT ERROR_MESSAGE() AS HataMesaji;
END CATCH;
```

```sql
--son ziyaret görüntüleme
-- Önce hastanın mevcut bilgilerine bakalım (SonZiyaretTarihi'ne dikkat)
SELECT * FROM Hastalar
WHERE HastaID = (SELECT HastaID FROM Randevular WHERE RandevuID = 40);

-- ŞİMDİ MUAYENEYİ GİRİYORUZ (Bu işlem trigger'ı çalıştırır)
INSERT INTO Muayeneler (RandevuID, Sikayet, Teshis, TedaviNotu)
VALUES (40, 'Test Baş Ağrısı', 'Migren', 'İlaç verildi');

-- SONUÇ KONTROLÜ
-- Hastanın SonZiyaretTarihi güncellendi mi? (Bugünün tarihi olmalı)
SELECT * FROM Hastalar
WHERE HastaID = (SELECT HastaID FROM Randevular WHERE RandevuID = 40);
```

5. Değerlendirme ve Sonuç

Bu proje sonucunda, RumeliCare Polikliniği'nin ihtiyaçlarını karşılayan, ölçeklenebilir ve güvenli bir veritabanı mimarisi oluşturulmuştur. Tasarımın **3NF** kurallarına uygunluğu veri tekrarını minimize etmiş, **Trigger** kullanımı operasyonel hataları (çakışan randevu vb.) sıfıra indirmiş ve **Stored Procedure** yapısı uygulama geliştirme sürecini hızlandırmıştır. Sistem, kliniğin dijital dönüşümünü başarıyla destekleyecek altyapıya sahiptir.

Kaynakça;

- BLG307 Veritabanı Yönetim Sistemleri Dönem Projesi Tanım Dokümanı.
- Reorting guru youtube - web sitesi
- Mobilhanem